

Lista de Exercícios 01

Para cada questão, implemente uma classe Java seguindo a especificação abaixo.

Regras gerais, válidas para todas as questões:

- Todos os atributos devem ser privados.
- Implemente getters públicos para todos os atributos.
- Implemente setters públicos apenas para os atributos indicados em cada questão (aqueles cujo valor precisa ser alterado por algum método da classe).
- Utilize os próprios getters e setters dentro dos métodos sempre que precisar ler ou alterar um atributo.
- Toda classe deve ter um método **main** com um exemplo de uso.

Questão 1 — Situação do Aluno (classe Aluno)

Uma escola quer automatizar o cálculo da situação final de um aluno, considerando não só o desempenho nas provas, mas também a frequência às aulas. Um aluno só é aprovado se atingir a média mínima e o mínimo de frequência ao mesmo tempo; quem fica abaixo em nota, em frequência, ou em ambos, recebe uma situação diferente que indica o motivo da reprovação.

Atributos: nota1, nota2, frequencia (percentual de 0 a 100).

Getters: para os três atributos.

Métodos:

- situacao() — calcula a média das duas notas e imprime a situação final, de acordo com as regras:
 - Média $\geq 6,0$ e frequência $\geq 75\%$ → "Situação: Aprovado"
 - Média $< 6,0$ e frequência $\geq 75\%$ → "Situação: Reprovado por Nota"
 - Média $\geq 6,0$ e frequência $< 75\%$ → "Situação: Reprovado por Falta"
 - Média $< 6,0$ e frequência $< 75\%$ → "Situação: Reprovado por Nota e Falta"

Questão 2 — Conta Bancária (classe ContaBancaria)

Um banco quer representar, de forma simplificada, uma conta corrente, guardando quem é o titular e o saldo disponível. A conta deve permitir depósitos, que aumentam o saldo, e saques, que só devem ser realizados quando há saldo suficiente; caso contrário, o saque é recusado e o cliente é avisado.

Atributos: titular, saldo.

Getters: para os dois atributos.

Setters: apenas para saldo.

Métodos:

- depositar(valor) — soma o valor recebido ao saldo atual.
- sacar(valor) — se houver saldo suficiente, caso contrário, imprime "Saldo insuficiente." e mantém o saldo inalterado.

Questão 3 — Contador Simples (classe Contador)

Este exercício simula um contador que exibe uma sequência de números até um limite definido pelo usuário, útil para praticar estruturas de repetição.

Atributo: limite.

Getter: para limite.

Métodos:

- contarAte() — imprime, um por linha, todos os números inteiros de 1 até limite (inclusive), usando uma estrutura de repetição.

Questão 4 — Controle de Estoque (classe Estoque)

Uma loja quer monitorar o estoque de seus produtos e simular o efeito das vendas diárias sobre a quantidade disponível, sinalizando quando um produto se aproxima do nível mínimo aceitável.

Atributos: nomeProduto, quantidadeMinima (quantidade mínima aceitável em estoque), quantidadeAtual.

Getters: para os três atributos.

Setters: apenas para quantidadeAtual (é alterada a cada dia durante a simulação de vendas).

Métodos:

- precisaReabastecer() — retorna true se a quantidade atual for menor que a quantidade mínima, e false caso contrário.
- simularVendas(dias, vendasPorDia) — simula, dia a dia, a venda de vendasPorDia unidades, imprimindo a quantidade em estoque ao final de cada dia no formato "Dia X - Estoque: Y". A simulação para assim que o número de dias informado for atingido ou o estoque chegar a zero (ou menos), o que ocorrer primeiro.

Questão 5 — Bônus de Funcionário (classe Funcionario)

Uma empresa quer calcular automaticamente o bônus de seus funcionários com base na faixa salarial em que cada um se encontra.

Atributos: nome, salario.

Getters: para os dois atributos.

Métodos:

- `calcularBonus()` — retorna o valor do bônus, de acordo com as regras:
 - Salário menor que R\$ 1.000,00 → bônus de 10% do salário
 - Salário entre R\$ 1.000,00 e R\$ 3.000,00 → bônus de 5% do salário
 - Salário acima de R\$ 3.000,00 → bônus de 2% do salário

Questão 6 — Matriz 3x3 (classe Matriz)

Este exercício trabalha a leitura e a exibição de dados organizados em uma grade quadrada de 3 linhas por 3 colunas, prática comum em manipulação de tabelas de valores.

Atributo: valores, uma matriz 3x3 de números.

Getter: para valores.

Métodos:

- `imprimir()` — exibe o conteúdo da matriz na tela, linha por linha, com os valores de cada linha separados por espaço.

No main: use a classe Scanner para ler, via teclado, os 9 valores da matriz (posição por posição) antes de instanciar o objeto e chamar `imprimir()`.

Questão 7 — Calculadora com Menu Interativo (classe MenuInterativo)

Este exercício simula uma calculadora simples operada por um menu, permitindo ao usuário escolher repetidamente qual operação deseja realizar até decidir encerrar o programa.

Atributos privados: `numero1`, `numero2`.

Não é necessário um construtor com parâmetros: os valores são definidos a cada operação escolhida no menu.

Getters e setters: para os dois atributos (os setters são usados no main para registrar os números informados pelo usuário antes de cada operação).

Métodos:

- `somar()`, `subtrair()`, `multiplicar()`, `dividir()` — cada um retorna o resultado da operação correspondente entre `numero1` e `numero2`.
- `dividir()` deve tratar a divisão por zero: se `numero2` for igual a zero, imprime "Não é possível dividir por zero." e retorna 0, sem executar a divisão.

No main: implemente um menu interativo com Scanner, exibido repetidamente (estrutura do-while) até que o usuário escolha sair. Opções do menu: 1 - Somar, 2 - Subtrair, 3 - Multiplicar, 4 - Dividir, 0 - Sair. Ao escolher uma operação válida (1 a 4), solicite os dois

números ao usuário antes de exibir o resultado. Se a opção digitada for inválida (diferente de 0 a 4), exiba "Opção inválida."

Questão 8 — Verificação de Número Primo (classe NumeroPrimo)

Este exercício verifica se um número informado é primo, conceito fundamental em teoria dos números e um bom exercício de lógica com laços de repetição.

Atributo privado: numero.

Getter: para numero.

Métodos:

- verificarPrimo() — retorna true caso o número seja primo, e false caso contrário. Considere que números menores que 2 nunca são primos.

Questão 9 — Maioridade (classe Pessoa)

Este exercício verifica se uma pessoa já atingiu a maioridade a partir da idade informada.

Atributos privados: nome, idade.

Getters: para os dois atributos.

Métodos:

- isMaiorDeIdade() — retorna true caso a idade seja maior ou igual a 18 anos, e false caso contrário.

Questão 10 — Cadastro de Produtos (classe Produto)

Uma loja quer manter um cadastro simples de produtos, reunindo nome, preço e quantidade disponível de cada item, para exibição em um relatório padronizado.

Atributos privados: nome, preco, quantidade.

Getters: para os três atributos.

No main: crie pelo menos dois objetos Produto distintos e exiba as informações de cada um no formato "nome - R\$preço - quantidade un."

Questão 11 — Área do Retângulo (classe Retangulo)

Este exercício calcula a área de um retângulo a partir de suas dimensões.

Atributos privados: largura, altura.

Getters: para os dois atributos.

Métodos:

- `calcularArea()` — retorna a área do retângulo (largura multiplicada pela altura).

Questão 12 — Validador de Senha Forte (classe `SenhaValidador`) Um sistema quer garantir que seus usuários cadastrem apenas senhas consideradas fortes, verificando um conjunto mínimo de critérios de segurança.

Atributo privado: senha.

Getter: para senha.

Métodos:

- `isSenhaForte()` — retorna `true` somente se todas as condições abaixo forem satisfeitas:
 - a senha possuir 8 ou mais caracteres;
 - a senha conter pelo menos um dígito numérico;
 - a senha conter pelo menos uma letra maiúscula.

Caso qualquer uma das condições não seja atendida, o método deve retornar `false`.

Questão 13 — Classificação de Triângulos (classe `Triangulo`)

Este exercício classifica um triângulo quanto à relação entre as medidas dos seus três lados.

Atributos privados: `ladoA`, `ladoB`, `ladoC`.

Getters: para os três atributos.

Métodos:

- `classificar()` — retorna a classificação do triângulo quanto aos lados, de acordo com as regras:
 - Os três lados iguais → "Equilátero"
 - Exatamente dois lados iguais → "Isósceles"
 - Nenhum lado igual → "Escaleno"